Module Interface Specification for Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

April 2, 2026

1 Revision History

Date	Version	Notes
Nov 6 - Nov 14	1.0	Preliminary Draft
March 17 - ?	2.0	Starting final draft

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at SRS

Contents

3 Introduction

The following document details the Module Interface Specifications for the PhysioCompanion app. The application is designed to identify users performing prescribed exercises and provide feedback on their form. This tool uses computer vision, specifically OpenCV and MediaPipe, to analyze user video recording and perform analysis. Please note that certain modules exhibit functional overlap, as their respective operations perform equivalent decomposed tasks across different modules.

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at here.

4 Notation

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003). The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1|c_2 \Rightarrow r_2|...|c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Physiotherapy Companion.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)
float	F	floating point numbers within (approx.) (-$1.798{*}10^{308}$, $1.798{*}10^{308}$)
object	X	an abstract representation of a collection of the above, similar to tuple
array	a	a collection of a single data type, n entries of the form (a_0, a_1, a_2, ... ,a_{n-1})
list	L	an array of non-fixed length

This specification of PhysioCompanion uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In

addition, PhysioCcompanion uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	**Level 2**
Hardware-Hiding Module	
Behaviour-Hiding Module	User Interface Module Home Module Main Module Draw Module
Software Decision Module	Database Management Module Generator Module Metrics Module Analyze Module Feedback Module

Table 1: Module Hierarchy

6 MIS

This section describes individual module specifications.
See Appendix A for information about the variables used throughout this document.

6.1 Module - Account Creation

This module enables the creation of an account for users. It uses a database to store any information that is essential for other modules to perform their actions. A database was used for its ease of manipulation and its comprehensive, multi-functional nature.
The database tables related to this module are:

1. User data (`users`): This table consists of information such as name, role, email, birthday, license_no/invite_code, createdAt and physioID.

2. Invite_codes (`inviteCodes`): This table contains all valid invite codes that will be used to validate patient account creation on the application.

3. Valid licenses (`validLicenses`): This is used for verification and validation purposes for a physiotherapist account creation.

6.1.1 Uses

All the variables used in each class related to account creation are described below.
/technically-functional/mobile/app/(auth)/create-physio.tsx,
/technically-functional/mobile/app/(auth)/create-patient.tsx
Variables used in both of the above files are listed below.

`const router = useRouter();`
: Expo Router instance for navigation after form submission. Imported from `expo-router`.

`const [name, setName] = useState("");`
: Manages the full name input field; initializes as empty string.

`const [email, setEmail] = useState("");`
: Manages the email address input field; initializes as empty string.

`const [password, setPassword] = useState("");`
: Manages the password input field; initializes as empty string.

`const [loading, setLoading] = useState(false);`
: Boolean flag that indicates form submission status. Set to `true` during API calls to disable inputs and show loading indicators.

`const [birthday, setBirthday] = useState("");`
: Manages the birthday input field; initializes as empty string; valid format is YYYY-MM-DD.

Unique variable in *create-patient.tsx*:

`const [inviteCode, setInviteCode] = useState("");`
: Manages the invitation code input field; initializes as empty string; valid format is 6 capital alphanumeric characters.

Unique variable in *create-physio.tsx*:

`const [licenseNumber, setLicenseNumber] = useState("");`
: Manages the physiotherapist license number input field; initializes as empty string; Required for physiotherapist verification; valid format is XX-123456.

/technically-functional/mobile/lib/authService.ts

`export const checkEmailExists`
: Async function constant that checks if an email is already registered in Firebase Authentication. Function accepts an email string as a parameter. Returns `true` if the email exists, `false` otherwise. Used to prevent duplicate account creation.

`export const licenseFormatValid`
: Function constant that validates physiotherapist license number format (XX-123456). Accepts a license number string as a parameter. Returns an object containing `isValid` (boolean) and `errorMessage` (string or null).

`export const registerPhysio`
: Async function constant that registers a new physiotherapist account. Type: `(params: {name, email, password, licenseNumber, birthday}) => Promise<{uid: string, role: "physio"}>`. Returns {`uid, role`} on success.

`export const registerPatient`
: Async function constant that registers a new patient account. Type: `(params: {name, email, password, inviteCode, birthday}) => Promise<{uid: string, role: "patient"}>`. Returns {`uid, role`} on success.

`export const login`
: Async function constant that authenticates a user. Type: `(email: string, password: string) => Promise<{uid: string, ...userData}>`. Returns user data from Firestore on success.

`export const logout`
: Async function constant that signs out the current user. Type: `() => Promise<void>`.

6.1.2 Syntax

Exported Constants
authService.ts

`export const registerPatient`
: Async function constant that registers a new patient account.

`export const registerPhysio`
: Async function constant that registers a new physiotherapist account.

`export const checkEmailExists`
: Async function constant that checks if an email address is already registered in the system.

`export const licenseFormatValid`
: Async function that validates the format of a physiotherapist license number.

create-patient.tsx*, *create-physio.tsx
NA

Exported Access Programs
create-patient.tsx
NA
create-physio.tsx
NA
authService.ts

Input	Output	Exceptions
checkEmailExists(email): Checks if an email is already registered in Firebase Authentication.		
`email: string` - User's email address	`Promise<boolean>` - Returns true if email already exists, false otherwise	`Error` - Throws error if there is an error checking email.
licenseFormatValid(licenseNumber): Validates physiotherapist license number format (XX-123456).		
`licenseNumber: string` - PT license number	`boolean` - Returns true if format is valid, false otherwise	`Error` - Throws error if there is an error checking licenseNumber.
registerPhysio: Registers a new physiotherapist account into the databese after license validation		
`params: {name: string, email: string, password: string, licenseNumber: string}`	`Promise<{uid: string, role: "physio"}>` - Returns user ID and role	`Error` - Invalid license format, license not verified, Firebase Auth error
registerPatient: Registers a new patient account into the database.		
`params: {name: string, email: string, password: string, inviteCode: string, birthday: string}`	`Promise<{uid: string, role: "patient"}>` - Returns user ID and role, marks invite code as used	`Error` - Invalid invite code, inactive/used code, missing physio link, or Firebase Auth error

Exported Access Programs
`p`: (name, email, password, licenseNumber, birthday)
`pt`: (name, email, password, inviteCode, birthday)
`user`: p or pt

`e`: user.email
`n`: user.name
`pw`: user.password
`lno`: user.licenseNumber
`code`: user.inviteCode
`bday`: user.birthday
`cat`: user.createdAt
`pid`: user.physioId (for `pt` only)
`v`: user.verified (for `p` only)
`u`: user.uid
`del`: user.deleted (for `pt` only)
db: `users` table in database
db': newer version of `users` table
$i, j, k, l, m, n \in \{0 - 9\}$
$a, b \in \{A - Z\}$

E(e): 'checkEmailExists(e)'
L(l): 'licenseFormatValid(l)'
Phy(p): 'registerPhysio(p)'
Pt(pt): 'registerPatient(pt)'

Function definitions:
$E(e) \implies e \in db.email$

$L(l) \implies \exists\ l \mid l = (a + b + '-' + \text{'i'} + \text{'j'} + \text{'k'} + \text{'l'} + \text{'m'} + \text{'n'}) \in validLicenses$

$Phy(p) \implies db' = db \cup \texttt{users} \mapsto \{uid \mapsto \{cat, e, lno, n, role :' physio', v\}\}$

$Pt(pt) \implies db' = db \cup \texttt{users} \mapsto \{uid \mapsto \{cat, del, e, code, n, pid, role :' patient'\}\}$

6.1.3 Semantics

State Variables
See here for a list of additional state variables.
`first`: user 1
`second`: user 2
`first_uid`: uid of user 1
`second_uid`: uid of user 2

State Invariant
$\forall\ first, second \in \texttt{db}, first\_uid \cap second\_uid = \emptyset$
All users have different uids.

6.1.4 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

6.1.5 Assumptions

- Patients with the same name have different birthdays.
- Only one person can access user information in the database at a time since many values that are exported are asynchronous values.

6.1.6 Access Routine Semantics

Exported access routines
See here for state variables being used.
checkEmailExists: E(e)
Pre-condition: e $\in$ *db.email*
Function: E(e) = true
Post-condition: Returns true if email already exists in database(`users`), false otherwise.
licenseFormatValid: L(l)
Pre-condition: l $\in$ *db.validLicenses*
Function: L(l) = true
Post-condition: Returns true if license exists in database(`validLicenses`), false otherwise.
registerPhysio: Phy(p)
Pre-condition: $(p.name \cap p.email \cap p.pw \cap p.lno \cap p.bday \neq \emptyset) \cap (L(p.lno)) \cap (\neg(E(p.email)))$
Function: Phy(p)
Post-condition: $p \in$ `db`
registerPatient: Pt(pt)
Pre-condition: $(p.name \cap p.email \cap p.pw \cap p.code \cap p.bday \neq \emptyset) \cap (\neg(E(p.email)))$
Function: Pt(pt)
Post-condition: $p \in$ `db`

6.1.7 Local Functions

create-patient.tsx

Input	Output	Exceptions
CreatePatient: Physiotherapist registration screen component.		
`props: none`	`JSX.Element` - Renders patient registration form with validation	`Validations performed:` • Password: min 6 chars + at least one number • Birthday: YYYY-MM-DD format • Email: existence check via checkEmailExists() • Invite code validation handled by registerPatient()

create-physio.tsx

Input	Output	Exceptions
CreatePhysio: Physiotherapist registration screen component.		
`props: none`	`JSX.Element` - Renders physiotherapist registration form with validation	`Validations performed:` • Password: min 6 chars + at least one number • Birthday: YYYY-MM-DD format • Email: existence check via checkEmailExists() • License number validation handled by registerPhysio()

Both files

Input	Output	Exceptions
passwordCheck(): Internal helper function for password validation.		
`params: {password: string}`	`string | null`	• Returns "Password must be at least 6 characters long" if length < 6 • Returns "Password must contain at least one number" if no numbers present • Returns null if valid
validateBirthday(): Internal helper function for birthday format validation.		
`params: {birthday: string}`	`string | null`	• Returns "Please enter birthday in YYYY-MM-DD format" if format entered is invalid • Returns null if valid

Note: CreatePatient and registerPatient have an important distinction although they seem similar. CreatePatient is the React component which manages the interface, while registerPatient handles the backend logic for making a new patient account. CreatePhysio and registerPhysio have the same distinction.

Local Routines (definitions)
`p`: (name, email, password, licenseNumber, birthday)
`pt`: (name, email, password, inviteCode, birthday)
`user`: p or pt
`e`: user.email
`n`: user.name
`pw`: user.password
`lno`: user.licenseNumber
`code`: user.inviteCode
`bday`: user.birthday
Pc(pw): 'passwordCheck(pw)'
Vb(bday): 'validateBirthday(bday)'

Function definitions:
$Pc(pw) \implies pw.length \geq 6 \cap \exists\ d \mid d \in \{0-9\} \cap d \in pw$
`Date` = 'YYYY-MM-DD' | 'YYYY' $\in$ {0000 - 9999} $\cap$ 'MM' $\in$ {01 - 12} $\cap$ 'DD' $\in$ {01 - 31}

$Vb(bday) \implies bday \in Date$
Note: See 6.13 for CreatePhysio and CreatePatient interface specifications.

6.1.8 Access Routine Semantics

6.2 Module - Valid user

This module's purpose is to check whether a potential user is authorized to create an account. For feasible implementation purposes, a file with hard coded invite codes and license numbers will be used. The actions accomplishing the module's purpose are within the registerPhysio and registerPatient functions.

6.2.1 Uses

For interface elements, see 6.13.

Input Parameters:

`params`	Input object containing patient registration data. Type: {`name: string, email: string, password: string, inviteCode: string, birthday: string`}.
`code`	Normalized invite code. Type: string. Computed as `params.inviteCode.trim().toUpperCase()`.
`inviteRef`	Firestore document reference to `inviteCodes/code`. Type: `DocumentReference`.
`inviteSnap`	Firestore document snapshot. Type: `DocumentSnapshot`.
`invite`	Data from invite code document. Contains fields: `active` (boolean), `used` (boolean), `physioId` (string).
`physioId`	Physiotherapist ID extracted from invite code. Type: string.
`cred`	Firebase Auth user credential. Type: `UserCredential`.
`uid`	Firebase Authentication unique user ID. Type: string.
`db`	Firestore database instance. Imported from `./firebase`.
`auth`	Firebase Authentication instance. Imported from `./firebase`.

For additional exported constants, see ***authService.ts*** exported constants in 6.1.2.
Exported Access Programs
Not applicable

6.2.2 Semantics

State Variables
See 6.13 for UI state variables.
`pUser`: a patient user type
`code`: a patient invite code
`lno`: a PT's license number
State Invariant
$code['used'] == true \implies \exists pUser \mid pUser.inviteCode == code$

Environment Variables
Not applicable

6.2.3 registerPatient: Internal Validation Steps

The function performs the following sequential validations on the invite code:

1. **Existence Check**: Query Firestore `inviteCodes` collection for document with ID = `normalize(inviteCode)`. If not found, throw "Invalid invite code."
2. **Active Check**: If `invite.active` $\neq$ `true`, throw "Invite code inactive."
3. **Usage Check**: If `invite.used = true`, throw "Invite code already used."
4. **Physio Link Check**: If `invite.physioId = null` or does not exist, throw "Invite code missing physio link."
5. **Extract physioId**: Store `invite.physioId` for use in patient document.

Only after all validations pass does the function proceed to:

- Create Firebase Auth user
- Create Firestore user document with `role: "patient"` and `physioId`
- Mark invite code as used with `usedBy: uid`

6.2.4 Uses

- *Name*: Name
- *role*: role
- *invite*: invite_code
- *license*: license_no

6.2.5 Syntax

Exported Constants

- *is_valid*: is_valid

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: This checks if a physiotherapist's license is valid			
license_check	*name* *license*	*is_valid*	FieldEmptyError InvalidCredsError
Description: This checks if a new patient's invite code is valid.			
invite_code_check	*name* *invite*	*is_valid*	FieldEmptyError InvalidCredsError

Table 6: Exported Access Routines

State Variables
A: user_account_p
B: user_account_pt
C: user_account_p
D: user_account_pt
I: the set of valid invite_code
V: the set of valid license_code
invite_a: invite_code for user A
license_b: license_no for user B
invite_c: invite_code for user C
license_d: license_no for user D
db: 'User Account'

State Invariant
$\forall a, b, c, d \in A, B, C, D \implies \forall\ license\_b, license\_d : license\_b, license\_d \in \{license\_no\}$,
$\forall\ invite\_a, invite\_c : invite\_a, invite\_c \in \{invite\_code\}$
All license numbers and invite codes belong to their overall respective sets.
$\forall\ license\_b \in \mathbb{Z}^+, \forall\ license\_d \in \mathbb{Z}^+ : 100000 \leq license\_b, license\_d \leq 999999, \cap\ license\_b \neq license\_d$
$\forall\ invite\_a \in \mathbb{Z}^+, \forall\ invite\_c \in \mathbb{Z}^+ : 100000 \leq invite\_a, invite\_c \leq 999999, \cap\ invite\_a \neq invite\_c$
All the license numbers and invite codes are unique, positive and 6 digits.
$is\_valid \in \{true, false\}$

6.2.6 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

6.2.7 Assumptions

- Invite codes are randomly generated.
- Invite codes and license numbers are unique.

6.2.8 Access Routine Semantics

license_check(license):
Input: license
Transition:
$\forall\ B \in db \Leftrightarrow \exists\ license\_b \in V$
Output: *is_valid*
invite_code_check(invite):
Input: invite

Transition:
$\forall\ A \in db \Leftrightarrow \exists\ invite\_a \in I$
Output: *is_valid*

6.2.9 Local Functions

NA

6.3 Module - User Account

This module's purpose is facilitate data manipulation within the user database tables. It was created as a separate component to simplify implementation.

Account creation generates a single instance of one of the following objects using variables in the table below.

useraccount.ts

`uid: string;` Unique identifier for the user.

`email: string;` User's email address for authentication and communication.

`name: string;` User's full name.

`role: "patient" | "physio";` User role designation. Determines which fields are applicable:

- `"patient"` - Receives care from a physiotherapist
- `"physio"` - Provides care as a physiotherapist

`birthday?: string;` User's date of birth. Optional field common to both roles.

`licenseNumber?: string;` Professional license number. **Physio only:** Required for physiotherapists during verification.

`verified?: boolean;` Account verification status. Optional flag common to both roles.

`physioId?: string;` Assigned physiotherapist identifier. **Patient only:** References the physio they are seeing.

`inviteCode?: string;` Registration invitation code. **Patient only:** Used during account creation.

`createdAt?: any;` Account creation timestamp. Optional field for auditing.

`updatedAt?: any;` Last update timestamp. Optional field for tracking modifications.

`deleted?: boolean;` Soft delete flag. Optional field indicating if the account is marked for deletion.

`deletedAt?: any;` Deletion timestamp. Optional field recording when soft delete occurred.

- **Patient object** (`user_acc_patient`)**:** {`uid, email, name, role: "patient", birthday?, verified?, physioId?, inviteCode?, createdAt?, updatedAt?, deleted?, deletedAt?`}
- **Physio object** (`user_acc_physio`)**:** {`uid, email, name, role: "physio", birthday?, licenseNumber?, verified?, createdAt?, updatedAt?, deleted?, deletedAt?`}
- **Note:** `physioId` and `inviteCode` are exclusive to patients; `licenseNumber` is exclusive to PTs.

6.3.1 Uses

- email
- password
- name
- birthday
- acc_id
- role

6.3.2 Syntax

Exported Constants
name: name
id: acc_id
birthday: birthday

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: retrieves user account information from the user table			
get_userdb_info[2]	*Name* *birthday(optional)*	*user_acc_p* *user_acc_pt*	UserNotFoundError DownloadError
Description: allows PT to delete a patient account from the system			
PTaccount_delete	*Name* *email*	NA	FieldEmptyError UserNotFoundError
Description: verifies if username and password match for authentication			
username_pw_match	*email* *password*	NA	LoginMatchError

Table 7: Exported Access Routines

[2] *For this function, the output depends on context. For example, if this function was going to be used in one which corresponds to the PT functionality of viewing a patient, their view would be restricted to patients under their care.*

State Variables
email: email
pw: password
name: name
id: acc_id
patient: user_acc_p ([*id*, *name*, *p*: role, *bday*: birthday, *email*, *pw*])
physio: user_acc_pt ([*id*, *name*, *pt*: role, *ptbday*: birthday, *email*, *pw*])
D(x): 'PT_account_delete(x)'
db: 'User Account'
name_has_id: db.Name $\mapsto$ {db.acc_id}
user_info: db.acc_id $\mapsto$ {db.Name, db.role, db.birthday, db.email}

State Invariant
$\exists$ *acc_id* | (*email* $\cap$ *pw* $\notin \emptyset$)

6.3.3 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

6.3.4 Assumptions

- If two people have the same name, then it is assumed they have different birthdays.
- No two users have the same invite codes.
- No two PTs have the same license numbers.

Note: All the modules after this point may have missing information, which will be defined for a future iteration.

6.3.5 Access Routine Semantics

get_userdb_info()

- Input: name, birthday(optional)
- Transition: NA
- Output: $\exists\ n \in db.Name \cup (\exists\ n \in db.Name \cap b \in db.birthday) \implies patient \in db \mid patient.Name = n \cap patient.birthday = b$

PTaccount_delete

- Input: name, email
- Transition:
- Output: $\exists\ n \in db.Name \cup (\exists\ n \in db.Name \cap b \in db.birthday) \implies patient \in db \mid patient.Name = n \cap patient.birthday = b$

6.3.6 Local Functions

NA

6.4 Module - Exercise Data

This module allows manipulation of a patient's exercise instructions data. Despite being a smaller module, the functions defined below

6.4.1 Uses

- name
- acc_id
- exercise
- sets
- reps
- rest_time

6.4.2 Syntax

Exported Constants
exercise: exercise
sets: sets
reps: reps
rest_time: rest_time

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: provides exercise instructions to user			
exerc_instruct	*user_acc_p* *exercise*	*ex_instruct*	UserNotFoundError DownloadError NoExerciseError
Description: allows physiotherapist to modify exercise parameters			
PT_modifies_exercise	*user_acc_p* *sets* *repetitions* *rest_time*	*ex_instruct*	UserNotFoundError DownloadError NoExerciseError

Table 8: Exported Access Routines

6.4.3 Semantics

State Variables
ex: exercise

sets: sets
reps: reps
rest_time: rest_time
ex_routine: [ex, sets, reps, rest_time]
name: name
acc_id: acc_id

State Invariant
$\forall x \in ex\_routine : x \notin \emptyset$
$\forall x \in \{acc\_id\} : \exists y \in \{ex\_routine\} \mid count(y) = 1$
$\forall x \in \{ex\_routine\} \mid x['sets'] = 1 \implies x['rest\_time'] = 0$

6.4.4 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

6.4.5 Assumptions

- During performance of an exercise, breaks may be needed, but the rest_time specified maybe 0. In this case, it is assumed that the recording takes the speed of performance into account and records for longer.
- For the same user, no two exercises with the same name are present.

6.4.6 Access Routine Semantics

6.4.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: provides exercise instructions to user			
add_ex	*name* *exercise* *acc_id* *sets* *reps* *rest_time*	*NA*	UserNotFoundError DownloadError NoExerciseError

Table 9: Local Routines

6.5 Module - User Activity

This module handles video data of a patient's recorded exercise(s).

6.5.1 Uses

- name
- exercise
- date
- duration
- recent_video
- recent_analysis
- overall_analysis

6.5.2 Syntax

Exported Constants
analysis: recent_analysis
recent_video: recent_video
analysis_report: analysis_report

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: adds analysis results to user activity history			
add_analysis_to _user_act	*user_acc_p* *recent_analysis*	NA	UserNotFoundError SaveError
Description: saves video recording for user account			
save_video	*user_acc_p* *recent_video*	NA	UserNotFoundError UploadError VideoTooShortError
Description: retrieves analysis data for a specific date			
get_analysis	*user_acc_p* *date*	*recent_analysis*	UserNotFoundError NoAnalysisError DownloadError
Description: Saves overall analysis (generated by Generator) to User Activity table.			
save_overall_analysis	*user_acc_p* *date*	*recent_analysis*	UserNotFoundError NoAnalysisError DownloadError

Table 10: Exported Routines

6.6 Module - Performance Statistics Generator

This module's purpose is to display the most recent report, an overall analysis of progress, and rehabilitation insights.

6.6.1 Syntax

Exported Constants
NA
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Generates health insights after assessing all past reports			
generate_insights	*all_reports*	*insights*	NoActivityError NoConnectionError
Description: Generates a specific report			
generate_report	*date* *time*	*recent_report*	CannotGenerateError NoActivityError
Description: Retrieves specific report from database			
get_report	*Date* *Time*	*Report*	NA

Table 11: Generator Access Routines

6.6.2 Semantics

State Variables
A: an instance of user_acc_p
E: an instance of recent_analysis
V: video
R: report
RR: recent_report

State Invariant
$\forall RR, \exists(A \cap V)$
$\forall R, \exists(A \cap V_i | i \geq 1..n, n \in 1..\mathbb{N})$

6.6.3 Environment Variables

NA

6.6.4 Assumptions

NA

6.6.5 Access Routine Semantics

TBD

6.7 Module - Profile Activity

This module displays past exercises, allows viewing of user profile, and directs to the Performance Statistics Generator module. PTs are able to comment on their patient's submissions.

6.7.1 Syntax

Exported Constants
NA

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Displays user profile information			
view_profile	*Name*	UserAccount_P OR UserAccount_PT	User_not_found Download_error
Description: Displays analysis report for user			
display_report	*Name* *email*	NA	required_field_empty patient_not_found
Description: Displays summary information			
display_summary	*Email* *Password*	NA	wrong_email_password
Description: Shows patient's past exercises			
p_past_exercises	*Email*	NA	no_activity
Description: Provides physical therapist feedback on patient			
PT_feedback_on_P	*PTcomment*	NA	User_not_found Download_error

Table 12: Home Exported Access Routines

6.7.2 Semantics

State Variables
NA
State Invariant
NA

6.7.3 Environment Variables

NA

6.7.4 Assumptions

TBD

6.7.5 Access Routine Semantics

TBD

6.7.6 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets profile from generator			
get_profile_gen	*NA*	UserAccount_P OR UserAccount_PT	Cannot_access_db

Table 13: Account Management Access Routines

6.8 Module - Analysis Control Flow

This module handles the video recording, placing markers on it and checking if the patient's form is appropriate for performing the exercise. It does this through the use of external libraries (notably OpenCV).

6.8.1 Uses

6.8.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Gets metrics from metrics module			
get_metrics	*NA*	*angle* *height* *duration*	DataRetrievalError
Description: Sends overall analysis to home module			
overall_analysis_home	*name* *birthday*	*overall_analysis*	DataRetrievalError AnalysisNotFoundError
Description: Sends real time processed frames to metrics (to be forwarded to Analysis)			
analysis_rt_frame	*processed_frame*	*instructions_ui*	CannotAnalyzeError NoInputError
Description: Performs overall analysis (given at end of recording)			
analysis_overall	*recent_video*	recent_analysis	InsufficientDataError ProcessError
Description: Module will provide corrections (received from Metrics) to UI (e.g. angle is too small)			
corrections_metrics	*rt_metrics*	*NA*	User_not_found Download_error
Description: Sends recent analysis to home module			
send_analysis_home	*NA*	recent_analysis	UserNotFoundError

Table 14: Access Routines for Main Module

Access Routine Name	Input	Output	Exceptions
Description: Calls draw to give markers of the leg			
set_markers	unprocessed_frame	processed_frame	InvalidFrameError

Table 15: Access Routines for Main Module (Continued)

6.8.3 Semantics

State Variables

State Invariant

6.8.4 Environment variables

6.8.5 Assumptions

6.8.6 Access Routine Semantics

6.8.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets starting form of user			
get_initial_form	*input_frame*	unprocessed_frame	InvalidFrameError

Table 16: Local Access Routines (Main)

6.9 Module - Analyze

6.9.1 Uses

6.9.2 Syntax

Exported Constants
Exported Access Programs

6.9.3 Semantics

State Variables
TBD
State Invariant
TBD

Access Routine Name	Input	Output	Exceptions
Description: gets metrics from metrics module			
latest_metrics	*rt_metrics*	*NA*	InsufficientDataError
Description: sends overall analysis to Feedback module			
send_analysis_to_feedback	*recent_analysis*	*NA*	AnalysisNotFoundError
Description: performs real time analysis			
analysis_rt	*processed_frames*	recent_analysis	User_not_found Download_error
Description: overall analysis (given at end of recording)			
analysis_overall	*video*	recent_analysis	ConnectionError InsufficientDataError
Description: module will correct form (eg. if angle is too small)			
adjust_metrics	*processed_frame*	rt_metrics	UndetectedFormError
Description: send overall analysis to Database module			
send_analysis_to_db	*overall_analysis*	*NA*	User_not_found Download_error

Table 17: Access Routines for Analysis Module

6.9.4 Environment variables

NA

6.9.5 Assumptions

6.9.6 Access Routine Semantics

6.9.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets initial landmarks from the starting position			
get_initial_landmarks	*NA*	UserAccount_P OR UserAccount_PT	User_not_found Download_error

Table 18: Access Routines for Analysis Module

6.10 Module - Feedback

This module is responsible for delivering the feedback (e.g. to adjust form) to the Main module.

6.10.1 Uses

6.10.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: feedback to user given after analysis			
ex_feedback_to_main	*feedback*	*NA*	FeedbackEmptyError
Description: instructions given by analysis to construct feedback that will be forwarded to the Main module			
given_instruct	*feedback_instructions*	*NA*	InsEmptyError

Table 19: Access Routines for Analysis Module

6.10.3 Semantics

State Variables
TODO
State Invariant
TODO

6.10.4 Assumptions

TODO

6.10.5 Access Routine Semantics

TODO

6.10.6 Local Functions

6.11 Module - Metrics

This module measures metrics (*angle, duration, height*) that are necessary to construct any corrections to the user's form.

TODO

Access Routine Name	Input	Output	Exceptions
Description: Constructs feedback based on instructions from Analyze module			
construct_feedback	*feedback_instructions*	*feedback*	InsEmptyError

Table 20: Feedback Access Routines

6.11.1 Uses

6.11.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: obtains a frame from Main module			
get_frame	*processed_frame*	NA	NoFrameError
Description: calculates metrics from frame provided by Main module			
calculate_frame	*processed_frame*	*angle* *height* *duration*	NoFrameError
Description: obtains metrics that need to be adjusted by the user (calls Analysis)			
fixed_frame	*NA*	*fixed_metrics*	NoFrameError

Table 21: Access Routines for Analysis Module

6.11.3 Semantics

State Variables

State Invariant

6.11.4 Assumptions

6.11.5 Access Routine Semantics

6.11.6 Local Functions

6.11.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: measures the angle between the starting and final plane			
measure_angle	*initial_plane* *final_plane*	*angle*	FrameEmptyError
Description: measures difference in initial and final height			
measure_height	*initial_height* *final_height*	*height*	FrameEmptyError
Description: measures duration of the performed exercise			
measure_duration	*initial_time* *final_time*	*duration*	FrameEmptyError

Table 22: Metrics Access Routines

6.12 Module - Draw

This module is responsible for drawing landmarks by using *MediaPipe* (external library).

6.12.1 Uses

- starting_frame
- processed_frames_list
- unprocessed_frames_list
- feedback_instructions

6.12.2 Syntax

Exported Constants

- *can_record*: good_to_record
- *fixing_instructions*: feedback_instructions
- *processed*: processed_frames_list

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Checks if the recording environment is appropriate (e.g., lighting, background)			
record_check	*starting_frame*	*good_to_record*	UserNotFound Error DownloadError
Description: Analyzes the user's starting form against a correct form model			
form_check	*processed_frames_list*	*feedback_instructions*	UserNotFound Error DownloadError
Description: Overlays pose estimation points onto the video feed for user feedback			
draw_points	*unprocessed_frames*	processed_frames_list	UserNotFound Error DownloadError

Table 23: Draw Access Routines

6.12.3 Semantics

State Variables
frames: unprocessed_frames_list

State Invariant

6.12.4 Assumptions

- User's complete form is in frame.
- The application will not draw points if the user does not comply with instructions given by *record_check*.

6.12.5 Access Routine Semantics

TODO

6.12.6 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Calls MediaPipe's draw functionality			
mpipe_draw	*unprocessed_frames*	*processed_frames_list*	MPbusyError

Table 24: Account Management Access Routines

6.13 Module - UI

NA

6.13.1 Uses

6.13.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
placeholder	*NA*	UserAccount_P OR UserAccount_PT	User_not_found Download_error

6.13.3 Semantics

TBD
State Variables

TODO
State Invariant
TODO

6.13.4 Assumptions

6.13.5 Access Routine Semantics

6.13.6 Local Functions

6.13.7 Local Functions

TODO

Access Routine Name	Input	Output	Exceptions
placeholder	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired

Table 26: UI Access Routines

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

7 Appendix A

For clarity and readability, the variables used throughout the document are provided below.

Table 27: Variable used throughout MIS

Name	Type	Description	Used in sections
name	String	Name of the user	7.1
role	String	Values can be PT or patient	7.1
email	String	Used as login ID	7.1
password	String	Set by user for login purposes	7.1
birthday	String	User's birthday for verification	7.1
license_no	Int	Verification of PT	7.1
invite_code	String	Given by PT to patient for registration	7.1
is_valid	Boolean	Used to check validity of user (invite_code, license_no)	?
acc_id	Int	Identifier assigned to app users	?
acc_double	Boolean	Indicates whether an account already exists in the system	?
date	String	Date (YYYYMMDD)	7.1
time	String	Time (s)	7.2
recent_analysis	ADT	Most recently generated analysis	7.1
recent_report	Record	Generated report of recent_video	TBD
recent_video	String (JSON)	Last video recorded by patient	TBD
video	String (JSON)	A video in the user database	TBD
report	Record	Report(s) sent to Generator module for analysis OR requested by patient	TBD
ex_instruct	String	Performance instructions of requested exercise	TBD
sets	String	Number of sets given by PT	7.1
repetitions	String	Repetitions of exercise per set	7.1
rest_time	String	Rest time between sets	7.1
leg_landmarks	Array	MediaPipe output of leg	7.4
processed_frame	Image	Video frame with overlaid markers	7.4
PT_comment	String	PT's comment on patient activity	7.3
overall_analysis	Abstract object	the overall analysis of all patient activity	7.4

Table 27: Variable used throughout MIS (continued)

Name	Type	Description	Used in sections
processed_frames_list	List(Image object)	list of processed_frames	7.3
feedback_instructions	Abstract Object	instructions given from analysis to feedback module	7.3
fixed_metrics	float	corrected metrics	7.3
initial/final_ plane/time/height	float	initial and final measurements	7.3
ins_ui	String	PT's comment on patient activity	TBD
unprocessed_frames	List(Image objects)	Unprocessed frames list	TBD
rt_metrics	List(of float)	Real-time metrics	TBD
input_frame	Image	Input frame	TBD
exercise	String	Selected exercise	TBD
insights	String	Generated insights	TBD
all_reports	List[report]	All reports are stored in a list	TBD
starting_frame	Image	The last frame before recording	TBD
good_to_record	String	A message informing user that the system is ready to record	TBD

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. What have you learned from implementing another team's module?

Group - Reflection

1. N/A

2. N/A

3. N/A

4. I think that our other documents such as the requirements, hazard analysis need to be slightly adjusted as a result of the design documents. With the requirements specifically some of them pertain to how the application design is and after completing our initial draft of the MG and MIS, the design does not necessarily line up with the requirements document. Further, some of the hazards considered should be changed to include a couple more areas for hazards to occur. Lastly for both of the documents, the assumptions should be changed to incorporated design changes that we had not considered during that deliverable.

5. Some limitations on the current system design and with the set scope include the accuracy of the pose estimation, the responsiveness of the application, and the limitation of stored exercises. With more resources, we would like to improve the accuracy of the pose estimation by either using a more advanced model or training our own model specifically for physiotherapy exercises. Further, we would like to improve the responsiveness of the application by optimizing our code and potentially using more powerful hardware to run the application. And finally, although explicity stated in our scope, we would like to include more exercises and potentially allow for custom exercises to be added by physiotherapists.

6. One alternative design solution we considered was making the application a desktop application rather than an android application. The benefit of this design is that it would allow for more powerful hardware to be used, which could improve the responsiveness and accuracy of the application. However, the tradeoff is a limitation to the accessibility of the application, as not all users may have access to a desktop computer with hardware capable of recording. We ultimately chose the android application design because it allows for greater accessibility and convenience for users, as they can use the application on their mobile devices which are easier to setup and come with recording hardware.

7. Implementing another team's module made it clear that another level of description is needed in order to effectively communicate the design and function of each module to developers. While implementing the Pixelation module for team 7, we found the description of their access program inputs and outputs lacking in detail. With sufficient context from the rest of the documentation we were able to produce what we believed to be the intended functionality of the module, but it required a lot of extra time and effort to interpret the design document. Additionally, because of this vague input and output description, there were no names given to input and output variables, which made it harder to stick to their coding conventions but also even harder to interpret the context of the module.

   There was a lack of detail which I feel resonates with our documentation as well. The purposes and usage of each module by other modules should be clearly defined in order to avoid confusion during implementation. Working on the final version of the documentation, we will make sure to include detailed descriptions of each access program's inputs and outputs, as well as any necessary context for understanding the module's function within the overall system.

   There was an overall description of modules and their functionality which provided a good high-level overview of each module, something we think should be included in our documentation as well. Additionally the use of assumptions to clarify certain design decisions was helpful, and while we don't believe it to be the correct section to provide context it was a good signifier of a well made assumption.

   Overall, this experience has taught us the importance of clear and detailed documentation in order to facilitate effective communication and collaboration between teams.

Matthew - Reflection

1. I think something that went well was our changes to overall workflow. One thing that we recognized was starting our document on google docs and then transferring our to sections to GitHub on the last two days the the deliverable was due led to some things going unchecked and the deliverable not turning out the way we had planned. Further it had also resulted in a 'panic' when we had to resolve merge conflicts between our sections soon to the deadline. Overall, we cut google docs for this deliverable and

mainly focused on our sections within GitHub and doing multiple small PRs within the deliverable timeline and that had gone quite well. There are a few days left but a majority of the document has at least been accounted for and hopefully this workflow works for our team.

2. I think one pain point I experienced was time management. I had suggested that since the computing team's workload was quite different to the documentation team's, we could take on small sections within the document with hopes of submitting it early into the project. This did partially go well but with focus split in two areas it was hard to focus on the development side of things for the POC demo. This may be something that improves with time but as of now is currently a work in progress as development on my end is not where I want it to be.

3. I think that our design decisions partially came from speaking with our stakeholder (Kevin Yu) but other decisions were made with typical application creation. For example, the high-level was talked through with Kevin to simulate the timing and feedback delivery that you would encounter during physiotherapy but things like reporting, logging-in as well as account creation were from looking at other applications and aspects that went well from that end.

4. Group Q

5. Group Q

6. Group Q

Vaisnavi - Reflection

1. Something that went well was how we worked on creating more constant pull requests (PRs) to avoid merge conflicts. This worked out well with everyone consistently pushing more PRs early on, allowing someone on the team to review and effectively merge it in. Overall, the organization and collaboration went very smoothly, and it allowed us to ultimately finish ahead of our internal deadline so we had enough time for editing and formatting.

2. I think one of the major pain points through this deliverable was trying to balance both the development side for the POC demo while also trying to contribute to the document. As the main sub-team for the POC demo, we did take on smaller sections, but it was difficult to find that balance in time for both at the start.

3. We believe that most of our major design decisions stemmed directly from our meetings with our stakeholder, Kevin Yu. Throughout the development of this deliverable, a number of ideas that we originally planned had to be changed or completely re-worked based on the feedback we received. In several cases, Kevin helped us recognize that certain features were either not feasible within the course timeline or did not align with

the true purpose and scope of our project. This was extremely helpful overall to have this input to tackle these decisions early on in the process.

4. Group Q
5. Group Q
6. Group Q

Maham - Reflection

1. Despite our (extremely) busy schedules, we made time to meet when we could and offer assistance to any member that needed it. In addition, all of us kept in contact and checked in with each other. I liked that none of us completely abandoned this project, but rather balanced it quite well. Also, the more frequent PR requests strategy (implemented for this deliverable) ended up being immensely helpful.
2. A pain point for me was that a lot of my time and energy was taken up by external tasks. These were, unfortunately, appointments I could not compromise on because they had been scheduled months in advance. I would have loved to give this project more of my focus, and I tried my best to self-organize, but there were some areas I fell short. I will attempt to refine my balancing act for the future.
3. The project is still in the early stages of development, and as a result, we do not have many concrete elements. However, some of developed ideas and design decisions was aided by Kevin, our supervisor. At this stage, we are constructing the foundation of the software, and that knowledge was supplied by him. Eventually, when we do have a functional product, we will want to loop in other stakeholders, e.g. patients, for modifications
4. Group Q
5. Group Q
6. Group Q

Cieran - Reflection

1. During this deliverable there was a need to agree upon implementation tactics. The team met and nailed down an approach for the development of the proof of concept, and I think everyone did a good job of ensuring their thoughts on what the system *was* got at least mentioned when developing the system.
2. I yet again sharked my teammates with procrastination. I have apologized profusely for my lack of initiative with this deliverable and have taken steps to ensure progress is far more consistent and meaningful in the future. As for this deliverable, I worked to catch up my own lack of effort towards the end of the it.

3. I think the design decisions, specifically around the system's allowance for physiotherapists to alter patient's exercise routines, were very influenced by meetings with our advisor. As a team, we wanted to ensure that the system was something wanted by both patients and physiotherapists; designing based off of requirements from past physiotherapy patients and our physiotherapist advisor was a must. Some design decisions, such as checking against public databases for the physiotherapist's identification number, were made in an attempt at security for users of the application.

4. GROUP Q

5. GROUP Q

6. GROUP Q